Lista5\Q1\src\L5Q1\Aluno.java

```java
package L5Q1;

public class Aluno extends Pessoa {

    public Aluno(String nome, String matricula) {
        super();
        this.setNome(nome);
        this.setMatricula(matricula);
    }

    public boolean pedirEmprestado(Livro livro) {
        if (getLivrosEmprestados().size() < 3) {
            getLivrosEmprestados().add(livro);
            return true;
        } else {
            return false;
        }
    }

    public double devolverLivro(Livro livro, double diasAtraso) {
        if (getLivrosEmprestados().remove(livro)) {
            return diasAtraso * 0.5;
        } else {
            return -1;
        }
    }

}

```

Lista5\Q1\src\L5Q1\Livro.java

```java
package L5Q1;

public class Livro {
    private String titulo;
    private String autor;

    public String getTitulo() {
        return titulo;
    }
    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }
    public String getAutor() {
        return autor;
    }
    public void setAutor(String autor) {
        this.autor = autor;
    }

    public Livro(String titulo, String autor) {
        this.titulo = titulo;
        this.autor = autor;
    }

}

```

Lista5\Q1\src\Main.java

```java
import java.util.Scanner;
import java.util.ArrayList;
import L5Q1.*;

public class Main {
    private static ArrayList<Professor> professores = new ArrayList<>();
    private static ArrayList<Aluno> alunos = new ArrayList<>();
    private static ArrayList<Livro> livros = new ArrayList<>();

    public static void main(String[] args) throws Exception {
        System.out.println("Sistema de gerenciamento bibliotecario");
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\nMenu de opcoes:");
            System.out.println("1. Cadastrar Professor");
            System.out.println("2. Cadastrar Aluno");
            System.out.println("3. Cadastrar Livro");
            System.out.println("4. Emprestar Livro");
            System.out.println("5. Devolver Livro");
            System.out.println("0. Sair");
            System.out.println("Escolha uma opcao:");
            int opcao = scanner.nextInt();

            if (opcao == 0) {
                System.out.println("Encerrando o programa...");
                break;
            }

            switch (opcao) {
                case 1:
                    System.out.println("Digite o nome do professor:");
                    String nomeProfessor = scanner.next();
                    System.out.println("Digite a matricula do professor:");
                    String matriculaProfessor = scanner.next();
                    System.out.println("O professor e doutor? (TRUE / FALSE):");
                    boolean isDoutor = scanner.nextBoolean();
                    Professor professor = new Professor(nomeProfessor, matriculaProfessor, isDoutor);
                    professores.add(professor);
                    System.out.println("Professor cadastrado com sucesso!");
                    break;
                case 2:
                    System.out.println("Digite o nome do aluno:");
                    String nomeAluno = scanner.next();
                    System.out.println("Digite a matricula do aluno:");
                    String matriculaAluno = scanner.next();
                    Aluno aluno = new Aluno(nomeAluno, matriculaAluno);
                    alunos.add(aluno);
                    System.out.println("Aluno cadastrado com sucesso!");
                    break;
                case 3:
```

```java
                    System.out.println("Digite o titulo do livro:");
                    String tituloLivro = scanner.next();
                    System.out.println("Digite o autor do livro:");
                    String autorLivro = scanner.next();
                    Livro livro = new Livro(tituloLivro, autorLivro);
                    livros.add(livro);
                    System.out.println("Livro cadastrado com sucesso!");
                    break;
                case 4:
                    System.out.println("Digite a matricula da pessoa:");
                    String matriculaPessoa = scanner.next();
                    System.out.println("Digite o titulo do livro:");
                    String tituloLivroEmprestado = scanner.next();

                    Livro livroEncontrado = encontrarLivroPorTitulo(tituloLivroEmprestado);

                    if (livroEncontrado == null) {
                        System.out.println("Livro nao encontrado!");
                        break;
                    }

                    Professor profEmprestimo = encontrarProfessorPorMatricula(matriculaPessoa);
                    if (profEmprestimo != null) {
                        if (profEmprestimo.pedirEmprestado(livroEncontrado)) {
                            System.out.println("Livro emprestado com sucesso!");
                        } else {
                            System.out.println("Limite de livros emprestados atingido!");
                        }
                    } else {
                        Aluno alunoEmprestimo = encontrarAlunoPorMatricula(matriculaPessoa);
                        if (alunoEmprestimo != null) {
                            if (alunoEmprestimo.pedirEmprestado(livroEncontrado)) {
                                System.out.println("Livro emprestado com sucesso!");
                            } else {
                                System.out.println("Limite de livros emprestados atingido!");
                            }
                        } else {
                            System.out.println("Pessoa nao encontrada!");
                        }
                    }
                    break;

                case 5:
                    System.out.println("Digite a matricula da pessoa:");
                    String matriculaPessoaDevolucao = scanner.next();
                    System.out.println("Digite o titulo do livro:");
                    String tituloLivroDevolucao = scanner.next();
                    System.out.println("Digite os dias de atraso:");
                    double diasAtraso = scanner.nextDouble();

```

```java
                    Livro livroDevolucao = encontrarLivroPorTitulo(tituloLivroDevolucao);

                    if (livroDevolucao == null) {
                        System.out.println("Livro nao encontrado!");
                        break;
                    }

                    Professor profDevolucao = encontrarProfessorPorMatricula(matriculaPessoaDevolucao);
                    if (profDevolucao != null) {
                        double multa = profDevolucao.devolverLivro(livroDevolucao, diasAtraso);
                        if (multa >= 0) {
                            System.out.println("Livro devolvido com sucesso! Multa: " + multa);
                        } else {
                            System.out.println("Livro nao encontrado!");
                        }
                    } else {
                        Aluno alunoDevolucao = encontrarAlunoPorMatricula(matriculaPessoaDevolucao);
                        if (alunoDevolucao != null) {
                            double multa = alunoDevolucao.devolverLivro(livroDevolucao, diasAtraso);
                            if (multa >= 0) {
                                System.out.println("Livro devolvido com sucesso! Multa: " + multa);
                            } else {
                                System.out.println("Livro nao encontrado!");
                            }
                        } else {
                            System.out.println("Pessoa nao encontrada!");
                        }
                    }
                    break;

                default:
                    System.out.println("Opcao invalida!");
            }
        }
        scanner.close();
    }

    private static Professor encontrarProfessorPorMatricula(String matricula) {
        for (Professor prof : professores) {
            if (prof.getMatricula().equals(matricula)) {
                return prof;
            }
        }
        return null;
    }

    private static Aluno encontrarAlunoPorMatricula(String matricula) {
        for (Aluno aluno : alunos) {
```

```
            if (aluno.getMatricula().equals(matricula)) {
                return aluno;
            }
        }
        return null;
    }

    private static Livro encontrarLivroPorTitulo(String titulo) {
        for (Livro livro : livros) {
            if (livro.getTitulo().equals(titulo)) {
                return livro;
            }
        }
        return null;
    }
}


```

Lista5\Q1\src\L5Q1\Pessoa.java

```java
package L5Q1;
import java.util.ArrayList;


public class Pessoa {
    private String nome;
    private String matricula;
    private ArrayList<Livro> livrosEmprestados;

    public Pessoa() {
        this.livrosEmprestados = new ArrayList<>();
    }

    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getMatricula() {
        return matricula;
    }
    public void setMatricula(String matricula) {
        this.matricula = matricula;
    }
    public ArrayList<Livro> getLivrosEmprestados() {
        return livrosEmprestados;
    }
    public void setLivrosEmprestados(ArrayList<Livro> livrosEmprestados) {
        this.livrosEmprestados = livrosEmprestados;
    }


}

```

Lista5\Q1\src\L5Q1\Professor.java

```java
package L5Q1;

public class Professor extends Pessoa {
    private boolean isDoutor;

    public boolean isDoutor() {
        return isDoutor;
    }

    public void setDoutor(boolean isDoutor) {
        this.isDoutor = isDoutor;
    }

    public boolean pedirEmprestado(Livro livro) {
        if (getLivrosEmprestados().size() < 4) {
            getLivrosEmprestados().add(livro);
            return true;
        } else {
            return false;
        }
    }

    public double devolverLivro(Livro livro, double diasAtraso) {
        if (getLivrosEmprestados().remove(livro)) {
            if (isDoutor) {
                return diasAtraso;
            } else {
                return diasAtraso * 0.8;
            }
        } else {
            return -1;
        }
    }

    public Professor(String nome, String matricula, boolean isDoutor) {
        super();
        this.setNome(nome);
        this.setMatricula(matricula);
        this.isDoutor = isDoutor;
    }

}

```

Lista5\Q2\src\L5Q2\Enfermeiro.java

```java
package L5Q2;

public class Enfermeiro extends Funcionario {

    private int horasExtrasTrabalhadas;

    public Enfermeiro(String nome, int horasExtrasTrabalhadas) {
        super(nome, 2000, "INTEGRAL");
        this.horasExtrasTrabalhadas = horasExtrasTrabalhadas;
    }

    @Override
    public double exibirSalario() {
        double salario = 2000;
        if (horasExtrasTrabalhadas > 0) {
            salario += horasExtrasTrabalhadas * 50;
        }
        return salario;
    }

}

```

Lista5\Q2\src\L5Q2\Funcionario.java

```java
package L5Q2;

public class Funcionario {
    private String nome;
    private double salario;
    private String turno;

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getTurno() {
        return turno;
    }

    public void setTurno(String turno) {
        this.turno = turno;
    }

    public double exibirSalario() {
        return salario;
    }

    public Funcionario(String nome, double salario, String turno) {
        this.nome = nome;
        this.salario = 1200;
        this.turno = "INTEGRAL";
    }

}

```

Lista5\Q2\src\L5Q2\Hospital.java

```java
package L5Q2;
import java.util.ArrayList;

public class Hospital {
    private Medico diretor;
    private ArrayList<Funcionario> funcionarios;
    private ArrayList<String> cirurgiasRealizadas;

    public Hospital(Medico diretor) {
        this.diretor = diretor;
        this.funcionarios = new ArrayList<>();
        this.cirurgiasRealizadas = new ArrayList<>();
    }

    public Medico getDiretor() {
        return diretor;
    }

    public void setDiretor(Medico diretor) {
        this.diretor = diretor;
    }

    public ArrayList<Funcionario> getFuncionarios() {
        return funcionarios;
    }

    public void setFuncionarios(ArrayList<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    public ArrayList<String> getCirurgiasRealizadas() {
        return cirurgiasRealizadas;
    }

    public void setCirurgiasRealizadas(ArrayList<String> cirurgiasRealizadas) {
        this.cirurgiasRealizadas = cirurgiasRealizadas;
    }

    public void contratarFuncionario(Funcionario funcionario) {
        if (diretor != null && "DIRETOR".equals(diretor.getCargo())) {
            funcionarios.add(funcionario);
            System.out.println("Funcionario contratado com sucesso!");
        } else {
            System.out.println("Erro: Apenas o diretor pode contratar funcionarios!");
        }
    }

    public void demitirFuncionario(Funcionario funcionario) {
        if (diretor != null && "DIRETOR".equals(diretor.getCargo())) {
            if (funcionarios.remove(funcionario)) {
                System.out.println("Funcionario demitido com sucesso!");
```

```
            } else {
                System.out.println("Erro: Funcionario nao encontrado!");
            }
        } else {
            System.out.println("Erro: Apenas o diretor pode demitir funcionarios!");
        }
    }

    public void realizarCirurgia(String cirurgia, Medico medico) {
        if (medico.isCirurgiao()) {
            cirurgiasRealizadas.add(cirurgia);
            System.out.println("Cirurgia '" + cirurgia + "' realizada com sucesso pelo Dr. " + medico);
        } else {
            System.out.println("Erro: O medico " + medico + " nao e um cirurgiao!");
        }
    }

}

```

Lista5\Q2\src\Main.java

```java
import java.util.Scanner;
import L5Q2.*;

public class Main {
    static Hospital hospital;

    public static void main(String[] args) throws Exception {
        System.out.println("Sistema de gerenciamento hospitalar");
        Scanner scanner = new Scanner(System.in);
        System.out.println("Digite o nome do diretor:");
        String nomeDiretor = scanner.nextLine();
        System.out.println("Medico diretor tem especialidade? 1 - Pediatra, 2 - Cirurgiao, 3 - Ambos, 0 - Nenhum");
        int especialidadeDiretor = scanner.nextInt();
        switch(especialidadeDiretor) {
            case 1:
                hospital = new Hospital(new Medico(nomeDiretor, "DIRETOR", true, false));
                break;
            case 2:
                hospital = new Hospital(new Medico(nomeDiretor, "DIRETOR", false, true));
                break;
            case 3:
                hospital = new Hospital(new Medico(nomeDiretor, "DIRETOR", true, true));
                break;
            default:
                hospital = new Hospital(new Medico(nomeDiretor, "DIRETOR", false, false));
        }
        System.out.println("Hospital criado com sucesso!");
        System.out.println("Menu de opcoes:");
        System.out.println("1. Contratar Funcionario");
        System.out.println("2. Demitir Funcionario");
        System.out.println("3. Realizar Cirurgia");
        System.out.println("0. Sair");
        while (true) {
            System.out.println("Escolha uma opcao:");
            int opcao = scanner.nextInt();
            if (opcao == 0) {
                System.out.println("Encerrando o programa...");
                break;
            }
            switch (opcao) {
                case 1:
                    System.out.println("Digite o nome do funcionario:");
                    String nomeFuncionario = scanner.next();
                    System.out.println("Digite o cargo do funcionario (1 - FUNCIONARIO GERAL, 2 - MEDICO, 3 - ENFERMEIRO):");
                    int cargoFuncionario = scanner.nextInt();
                    switch (cargoFuncionario) {
                        case 1:
                            Funcionario funcionario = new Funcionario(nomeFuncionario, 1200, "INTEGRAL");
                            hospital.contratarFuncionario(funcionario);
```

```
                            break;
                        case 2:
                            System.out.println("O medico e estagiario ou formado? 1 - Estagiario, 2 - Formado");
                            int tipoMedico = scanner.nextInt();
                            if (tipoMedico == 1) {
                                Medico medicoEstagiario = new Medico(nomeFuncionario, "ESTAGIARIO", false, false);
                                hospital.contratarFuncionario(medicoEstagiario);
                            } else {
                                System.out.println("O medico tem especialidade? 1 - Pediatra, 2 - Cirurgiao, 3 - Ambos, 0 - Nenhum");
                                int especialidadeMedico = scanner.nextInt();
                                switch (especialidadeMedico) {
                                    case 1:
                                        hospital.contratarFuncionario(new Medico(nomeFuncionario, "MEDICO", true, false));
                                        break;
                                    case 2:
                                        hospital.contratarFuncionario(new Medico(nomeFuncionario, "MEDICO", false, true));
                                        break;
                                    case 3:
                                        hospital.contratarFuncionario(new Medico(nomeFuncionario, "MEDICO", true, true));
                                        break;
                                    default:
                                        hospital.contratarFuncionario(new Medico(nomeFuncionario, "MEDICO", false, false));
                                }
                            }
                            break;
                        case 3:
                            System.out.println("Digite as horas extras trabalhadas pelo enfermeiro:");
                            int horasExtras = scanner.nextInt();
                            Enfermeiro enfermeiro = new Enfermeiro(nomeFuncionario, horasExtras);
                            hospital.contratarFuncionario(enfermeiro);
                            break;
                        default:
                            System.out.println("Cargo invalido! Tente novamente.");
                    }
                    break;
                case 2:
                    System.out.println("Digite o nome do funcionario a ser demitido:");
                    String nomeFuncionarioDemitir = scanner.next();
                    Funcionario funcionario = encontrarFuncionario-PorNome(nomeFuncionarioDemitir);
                    if (funcionario != null) {
                        hospital.demitirFuncionario(funcionario);
                    } else {
                        System.out.println("Funcionario nao encontrado! Tente novamente.");
                    }
```

```java

                    break;
                case 3:
                    System.out.println("Digite o nome do medico que realizara a cirurgia:");
                    String nomeMedicoCirurgia = scanner.next();
                    Funcionario medicoCirurgia = encontrarFuncionario-PorNome(nomeMedicoCirurgia);
                    if (medicoCirurgia != null && medicoCirurgia instanceof Medico) {
                        System.out.println("Digite o nome da cirurgia a ser realizada:");
                        String nomeCirurgia = scanner.next();
                        hospital.realizarCirurgia(nomeCirurgia, (Medico) medicoCirurgia);
                    } else {
                        System.out.println("Medico nao encontrado ou funcionario nao e cirurgiao! Tente novamente.");
                    }
                    break;
                default:
                    System.out.println("Opcao invalida! Tente novamente.");
            }
        }
        scanner.close();
    }
    static Funcionario encontrarFuncionarioPorNome(String nome) {
        for (Funcionario funcionario : hospital.getFuncionarios()) {
            if (funcionario.getNome().equals(nome)) {
                return funcionario;
            }
        }
        return null;
    }
}

```

Lista5\Q2\src\L5Q2\Medico.java

```java
package L5Q2;

public class Medico extends Funcionario {

    private String cargo;
    private boolean isPediatra;
    private boolean isCirurgiao;
    private Medico assistenteEstagiario;
    private Enfermeiro assistenteEnfermeiro;

    public Medico(String nome, String cargo, boolean isPediatra, boolean isCirurgiao) {
        super(nome, 1200, "INTEGRAL");
        this.cargo = cargo;
        this.isPediatra = isPediatra;
        this.isCirurgiao = isCirurgiao;
        this.assistenteEstagiario = null;
        this.assistenteEnfermeiro = null;
    }

    public boolean atribuirAssistenteEstagiario(Medico medico) {
        if ("ESTAGIARIO".equals(this.cargo)) {
            System.out.println("Erro: Medico estagiario nao pode ter assistente!");
            return false;
        }

        if (medico == null || !"ESTAGIARIO".equals(medico.cargo)) {
            System.out.println("Erro: O assistente deve ser um medico estagiario!");
            return false;
        }

        this.assistenteEstagiario = medico;
        System.out.println("Assistente estagiario atribuido com sucesso!");
        return true;
    }

    public boolean atribuirAssistenteEnfermeiro(Enfermeiro enfermeiro) {
        if ("ESTAGIARIO".equals(this.cargo)) {
            System.out.println("Erro: Medico estagiario nao pode ter assistente!");
            return false;
        }

        if (enfermeiro == null) {
            System.out.println("Erro: Enfermeiro invalido!");
            return false;
        }

        this.assistenteEnfermeiro = enfermeiro;
        System.out.println("Assistente enfermeiro atribuido com sucesso!");
        return true;
    }

```

```java
    public Medico getAssistenteEstagiario() {
        return assistenteEstagiario;
    }

    public Enfermeiro getAssistenteEnfermeiro() {
        return assistenteEnfermeiro;
    }

    public void contratarFuncionario(Funcionario funcionario) {
        if ("DIRETOR".equals(this.cargo)) {
            System.out.println("Funcionario contratado com sucesso!");
        } else {
            System.out.println("Erro: Apenas medicos com cargo de Diretor podem contratar funcionarios!");
        }
    }

    public void demitirFuncionario(Funcionario funcionario) {
        if ("DIRETOR".equals(this.cargo)) {
            System.out.println("Funcionario demitido com sucesso!");
        } else {
            System.out.println("Erro: Apenas medicos com cargo de Diretor podem demitir funcionarios!");
        }
    }

    public String getCargo() {
        return cargo;
    }

    public void setCargo(String cargo) {
        this.cargo = cargo;
    }

    public boolean isPediatra() {
        return isPediatra;
    }

    public void setPediatra(boolean isPediatra) {
        this.isPediatra = isPediatra;
    }

    public boolean isCirurgiao() {
        return isCirurgiao;
    }

    public void setCirurgiao(boolean isCirurgiao) {
        this.isCirurgiao = isCirurgiao;
    }

    public void setAssistenteEstagiario(Medico assistenteEstagiario) {
        this.assistenteEstagiario = assistenteEstagiario;
    }

```

```java
    public void setAssistenteEnfermeiro(Enfermeiro assistenteEnfermeiro) {
        this.assistenteEnfermeiro = assistenteEnfermeiro;
    }

    @Override
    public double exibirSalario() {
        double salario = 1200;

        if ("ESTAGIARIO".equals(cargo)) {
            salario = 1200;
        } else if ("FORMADO".equals(cargo)) {
            salario = 5000;
        } else if ("DIRETOR".equals(cargo)) {
            salario = 10000;
        }

        if (isPediatra) {
            salario += 1000;
        }

        if (isCirurgiao) {
            salario += 1500;
        }

        return salario;
    }
}

```

Lista5\Q3\src\L5Q3\Aluno.java

```java
package L5Q3;

public class Aluno {
    private String nome;
    private String matricula;

    public Aluno(String nome, String matricula) {
        this.nome = nome;
        this.matricula = matricula;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getMatricula() {
        return matricula;
    }

    public void setMatricula(String matricula) {
        this.matricula = matricula;
    }

}

```

Lista5\Q3\src\L5Q3\AlunoFundamental.java

```java
package L5Q3;

public class AlunoFundamental extends Aluno {
    private Livro livroEmprestado;

    public AlunoFundamental(String nome, String matricula) {
        super(nome, matricula);
    }

    public Livro getLivroEmprestado() {
        return livroEmprestado;
    }

    public void setLivroEmprestado(Livro livroEmprestado) {
        this.livroEmprestado = livroEmprestado;
    }

}

```

Lista5\Q3\src\L5Q3\AlunoMedio.java

```java
package L5Q3;

import java.util.ArrayList;

public class AlunoMedio extends Aluno {
    private ArrayList<Livro> livrosEmprestados;

    public AlunoMedio(String nome, String matricula) {
        super(nome, matricula);
        this.livrosEmprestados = new ArrayList<>();
    }

    public ArrayList<Livro> getLivrosEmprestados() {
        return livrosEmprestados;
    }

    public void setLivrosEmprestados(ArrayList<Livro> livrosEmprestados) {
        this.livrosEmprestados = livrosEmprestados;
    }

}

```

Lista5\Q3\src\L5Q3\Biblioteca.java

```java
package L5Q3;
import java.util.ArrayList;

public class Biblioteca {
    private ArrayList<Livro> livrosDisponiveis;
    private ArrayList<Aluno> alunos;
    private ArrayList<String> historico;

    public Biblioteca() {
        this.livrosDisponiveis = new ArrayList<>();
        this.alunos = new ArrayList<>();
        this.historico = new ArrayList<>();
    }

    public void adicionarLivro(Livro livro) {
        livrosDisponiveis.add(livro);
        System.out.println("Livro '" + livro.getNome() + "' adicionado a biblioteca.");
    }

    public void registrarAluno(Aluno aluno) {
        if (buscarMatricula(aluno.getMatricula()) == null) {
            alunos.add(aluno);
            System.out.println("Aluno registrado com sucesso!");
        } else {
            System.out.println("Erro: Aluno com essa matricula ja registrado.");
        }
    }

    public void emprestarLivro(String nomeAluno, String nomeLivro) {
        ArrayList<Aluno> alunosComNome = buscarNomeMultiplos(nomeAluno);

        if (alunosComNome.isEmpty()) {
            System.out.println("Erro: Aluno nao encontrado.");
            return;
        }

        if (alunosComNome.size() > 1) {
            System.out.println("Erro: Multiplos alunos com esse nome. Use o metodo com matricula.");
            return;
        }

        Aluno aluno = alunosComNome.get(0);
        emprestarLivroInterno(aluno, nomeLivro);
    }

    public void emprestarLivro(String nomeAluno, String matricula, String nomeLivro) {
        Aluno aluno = buscarMatricula(matricula);

        if (aluno == null) {
            System.out.println("Erro: Aluno com essa matricula nao encontrado.");
            return;
```

```java
        }

        if (!aluno.getNome().equals(nomeAluno)) {
            System.out.println("Erro: Matricula nao corresponde ao nome fornecido.");
            return;
        }

        emprestarLivroInterno(aluno, nomeLivro);
    }

    private void emprestarLivroInterno(Aluno aluno, String nomeLivro) {
        if (aluno instanceof AlunoFundamental) {
            AlunoFundamental alunoFund = (AlunoFundamental) aluno;
            if (alunoFund.getLivroEmprestado() != null) {
                System.out.println("Erro: Aluno do ensino fundamental ja possui um livro emprestado. Devolva o livro anterior.");
                return;
            }
        }

        Livro livro = buscarLivro(nomeLivro);
        if (livro == null) {
            System.out.println("Erro: Livro nao disponivel na biblioteca.");
            return;
        }

        livrosDisponiveis.remove(livro);

        if (aluno instanceof AlunoFundamental) {
            ((AlunoFundamental) aluno).setLivroEmprestado(livro);
        } else if (aluno instanceof AlunoMedio) {
            ((AlunoMedio) aluno).getLivrosEmprestados().add(livro);
        }

        historico.add(aluno.getNome() + " (Matricula: " + aluno.getMatricula() + ") emprestou '" + livro.getNome() + "'");
        System.out.println("Livro '" + livro.getNome() + "' emprestado com sucesso!");
    }

    public void receberLivro(String nomeAluno, String nomeLivro) {
        ArrayList<Aluno> alunosComNome = buscarNomeMultiplos(nomeAluno);

        if (alunosComNome.isEmpty()) {
            System.out.println("Erro: Aluno nao encontrado.");
            return;
        }

        if (alunosComNome.size() > 1) {
            System.out.println("Erro: Multiplos alunos com esse nome. Use o metodo com matricula.");
            return;
        }

        Aluno aluno = alunosComNome.get(0);
```

```java
        receberLivroInterno(aluno, nomeLivro);
    }

    public void receberLivro(String nomeAluno, String matricula, String nomeLivro) {
        Aluno aluno = buscarMatricula(matricula);

        if (aluno == null) {
            System.out.println("Erro: Aluno com essa matricula nao encontrado.");
            return;
        }

        if (!aluno.getNome().equals(nomeAluno)) {
            System.out.println("Erro: Matricula nao corresponde ao nome fornecido.");
            return;
        }

        receberLivroInterno(aluno, nomeLivro);
    }

    private void receberLivroInterno(Aluno aluno, String nomeLivro) {
        Livro livro = null;
        if (aluno instanceof AlunoMedio) {
            ArrayList<Livro> livros = ((AlunoMedio) aluno).getLivrosEmprestados();
            for (Livro l : livros) {
                if (l.getNome().equals(nomeLivro)) {
                    livro = l;
                    livros.remove(l);
                    break;
                }
            }
        } else if (aluno instanceof AlunoFundamental) {
            AlunoFundamental alunoFund = (AlunoFundamental) aluno;
            if (alunoFund.getLivroEmprestado() != null &&
alunoFund.getLivroEmprestado().getNome().equals(nomeLivro)) {
                livro = alunoFund.getLivroEmprestado();
                alunoFund.setLivroEmprestado(null);
            }
        }

        if (livro == null) {
            System.out.println("Erro: Este aluno nao tem este livro emprestado.");
            return;
        }

        livrosDisponiveis.add(livro);
        historico.add(aluno.getNome() + " (Matricula: " + aluno.getMatricula() + ")
devolveu '" + livro.getNome() + "'");
        System.out.println("Livro '" + livro.getNome() + "' devolvido com sucesso!");
    }

    public void mostrarAlunosRegistrados() {
        if (alunos.isEmpty()) {
            System.out.println("Nenhum aluno registrado.");
            return;
```

```java
        }
        System.out.println("Alunos Registrados");
        for (Aluno aluno : alunos) {
            String tipo = (aluno instanceof AlunoFundamental) ? "Fundamental" : "Medio";
            System.out.println("Nome: " + aluno.getNome() + " | Matricula: " + aluno.getMatricula() + " | Tipo: " + tipo);
        }
    }

    public void mostrarLivros() {
        System.out.println("Livros Disponiveis");
        if (livrosDisponiveis.isEmpty()) {
            System.out.println("Nenhum livro disponivel no momento.");
        } else {
            for (Livro livro : livrosDisponiveis) {
                System.out.println("Titulo: " + livro.getNome() + " | Autor: " + livro.getAutor());
            }
        }

        System.out.println("Livros Emprestados:");
        boolean temEmprestados = false;
        for (Aluno aluno : alunos) {
            if (aluno instanceof AlunoMedio) {
                ArrayList<Livro> livros = ((AlunoMedio) aluno).getLivrosEmprestados();
                for (Livro livro : livros) {
                    System.out.println("'" + livro.getNome() + "' - emprestado para " + aluno.getNome());
                    temEmprestados = true;
                }
            } else if (aluno instanceof AlunoFundamental) {
                AlunoFundamental alunoFund = (AlunoFundamental) aluno;
                if (alunoFund.getLivroEmprestado() != null) {
                    System.out.println("'" + alunoFund.getLivroEmprestado().getNome() + "' - emprestado para " + aluno.getNome());
                    temEmprestados = true;
                }
            }
        }
        if (!temEmprestados) {
            System.out.println("Nenhum livro emprestado.");
        }
    }

    public Aluno buscarMatricula(String matricula) {
        for (Aluno aluno : alunos) {
            if (aluno.getMatricula().equals(matricula)) {
                return aluno;
            }
        }
        return null;
    }

    public ArrayList<Aluno> buscarNomeMultiplos(String nome) {
```

```java
        ArrayList<Aluno> resultado = new ArrayList<>();
        for (Aluno aluno : alunos) {
            if (aluno.getNome().equals(nome)) {
                resultado.add(aluno);
            }
        }
        return resultado;
    }

    public Livro buscarLivro(String nome) {
        for (Livro livro : livrosDisponiveis) {
            if (livro.getNome().equals(nome)) {
                return livro;
            }
        }
        return null;
    }


    public void mostrarHistorico() {
        System.out.println("Historico de transacoes:");
        if (historico.isEmpty()) {
            System.out.println("Historico vazio.");
        } else {
            for (String transacao : historico) {
                System.out.println(transacao);
            }
        }
    }
}

```

Lista5\Q3\src\L5Q3\Livro.java

```java
package L5Q3;

public class Livro {
    private String nome;
    private String autor;


    public String getNome() {
        return nome;
    }



    public void setNome(String nome) {
        this.nome = nome;
    }



    public String getAutor() {
        return autor;
    }



    public void setAutor(String autor) {
        this.autor = autor;
    }



    public Livro(String nome, String autor) {
        this.nome = nome;
        this.autor = autor;
    }

}

```

Lista5\Q3\src\Main.java

```java
import L5Q3.*;
import java.util.Scanner;
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) throws Exception {
        Biblioteca biblioteca = new Biblioteca();
        Scanner scanner = new Scanner(System.in);

        boolean menu = true;
        while (menu) {
            System.out.println("Sistema de gerenciamento bibliotecario");
            System.out.println("1. Registrar Aluno");
            System.out.println("2. Registrar Livro");
            System.out.println("3. Emprestar Livro");
            System.out.println("4. Devolver Livro");
            System.out.println("5. Mostrar Alunos Registrados");
            System.out.println("6. Mostrar Livros");
            System.out.println("7. Mostrar Historico");
            System.out.println("0. Sair");
            System.out.println("Escolha uma opcao:");
            int opcao = scanner.nextInt();
            scanner.nextLine();

            switch (opcao) {
                case 1:
                    System.out.println("\nDigite o nome do aluno:");
                    String nome = scanner.nextLine();
                    System.out.println("Digite a matricula:");
                    String matricula = scanner.nextLine();
                    System.out.println("Tipo de aluno (1 = Fundamental, 2 = Medio):");
                    int tipo = scanner.nextInt();
                    scanner.nextLine();

                    Aluno novoAluno;
                    if (tipo == 1) {
                        novoAluno = new AlunoFundamental(nome, matricula);
                    } else {
                        novoAluno = new AlunoMedio(nome, matricula);
                    }
                    biblioteca.registrarAluno(novoAluno);
                    break;

                case 2:
                    System.out.println("\nDigite o titulo do livro:");
                    String titulo = scanner.nextLine();
                    System.out.println("Digite o autor do livro:");
                    String autor = scanner.nextLine();
                    biblioteca.adicionarLivro(new Livro(titulo, autor));
                    break;

```

```java
                case 3:
                    System.out.println("\nDigite o nome do aluno:");
                    String nomeAluno = scanner.nextLine();
                    System.out.println("Digite o nome do livro:");
                    String nomeLivro = scanner.nextLine();

                    ArrayList<Aluno> alunosEncontrados = biblioteca.buscarNomeMultiplos(nomeAluno);

                    if (alunosEncontrados.size() > 1) {
                        System.out.println("Multiplos alunos com esse nome. Digite a matricula:");
                        String matriculaEmprrestimo = scanner.nextLine();
                        biblioteca.emprestarLivro(nomeAluno, matriculaEmprrestimo, nomeLivro);
                    } else {
                        biblioteca.emprestarLivro(nomeAluno, nomeLivro);
                    }
                    break;

                case 4:
                    System.out.println("\nDigite o nome do aluno:");
                    String nomeAlunoDev = scanner.nextLine();
                    System.out.println("Digite o nome do livro:");
                    String nomeLivroDev = scanner.nextLine();

                    ArrayList<Aluno> alunosEncontradosDev = biblioteca.buscarNomeMultiplos(nomeAlunoDev);

                    if (alunosEncontradosDev.size() > 1) {
                        System.out.println("Multiplos alunos com esse nome. Digite a matricula:");
                        String matriculaDevolucao = scanner.nextLine();
                        biblioteca.receberLivro(nomeAlunoDev, matriculaDevolucao, nomeLivroDev);
                    } else {
                        biblioteca.receberLivro(nomeAlunoDev, nomeLivroDev);
                    }
                    break;

                case 5:
                    biblioteca.mostrarAlunosRegistrados();
                    break;

                case 6:
                    biblioteca.mostrarLivros();
                    break;

                case 7:
                    biblioteca.mostrarHistorico();
                    break;

                case 0:
                    System.out.println("Encerrando o programa...");
```

```
                    menu = false;
                    break;

                default:
                    System.out.println("Opcao invalida!");
            }
        }
        scanner.close();
    }
}

```

Lista5\Q4\src\L5Q4\ColetaMetal.java

```java
package L5Q4;

public class ColetaMetal extends PontoDeColeta {

    public ColetaMetal(String nome, int p) {
        super(nome, p);

    }

    @Override
    public void definirPreco(int p, String demandaAtual) {
        if (demandaAtual.equalsIgnoreCase("PAPELAO")) {
            super.setP(p * 6);
        } else if (demandaAtual.equalsIgnoreCase("METAL")) {
            super.setP(p * 20);
        } else {
            super.setP(p * 10);
        }
    }
}

```

Lista5\Q4\src\L5Q4\ColetaPapelao.java

```java
package L5Q4;

public class ColetaPapelao extends PontoDeColeta {

    public ColetaPapelao(String nome, int p) {
        super(nome, p);
    }

    @Override
    public void definirPreco(int p, String demandaAtual) {
        if (demandaAtual.equalsIgnoreCase("VIDRO")) {
            super.setP(p * 4);
        } else if (demandaAtual.equalsIgnoreCase("PAPELAO")) {
            super.setP(p * 14);
        } else {
            super.setP(p * 7);
        }

    }

}

```

Lista5\Q4\src\L5Q4\ColetaVidro.java

```java
package L5Q4;

public class ColetaVidro extends PontoDeColeta {

    public ColetaVidro(String nome, int p) {
        super(nome, p);
    }

    @Override
    public void definirPreco(int p, String demandaAtual) {
        if (demandaAtual.equalsIgnoreCase("METAL")) {
            super.setP(p * 3);
        } else if (demandaAtual.equalsIgnoreCase("VIDRO")) {
            super.setP(p * 10);
        } else {
            super.setP(p * 5);
        }

    }

}

```

Lista5\Q4\src\L5Q4\ListaDeEstabelecimentos.java

```java
package L5Q4;
import java.util.ArrayList;

public class ListaDeEstabelecimentos {
    private ArrayList<PontoDeColeta> pontos;
    private String demandaAtual;

    public ListaDeEstabelecimentos() {
        this.pontos = new ArrayList<>();
    }

    public void registrarEstabelecimento(PontoDeColeta ponto) {
        if (buscarEstabelecimento(ponto.getNome()) == null) {
            pontos.add(ponto);
            System.out.println("Ponto de Coleta registrado: " + ponto.getNome());
        } else if (verificarEstabelecimento(ponto)) {
            System.out.println("Ponto de Coleta do mesmo tipo ja registrado: " + ponto.getNome());
        }
    }

    public void removerEstabelecimento(String nome) {
        PontoDeColeta ponto = buscarEstabelecimento(nome);
        if (ponto != null) {
            pontos.remove(ponto);
            System.out.println("Ponto de Coleta removido: " + nome);
        }
    }

    public void definirDemanda(String demanda) {
        if (demanda.equals("VIDRO") || demanda.equals("PAPELAO") || demanda.equals("METAL")) {
            this.demandaAtual = demanda;
            System.out.println("Demanda definida: " + demanda);
        } else {
            System.out.println("Demanda invalida. Use VIDRO, PAPELAO ou METAL.");
            return;
        };
    }

    public void mostrarLista() {
        if (pontos.isEmpty()) {
            System.out.println("Nenhum ponto de coleta registrado.");
        } else {
            System.out.println("Pontos de Coleta Registrados:");
            for (PontoDeColeta ponto : pontos) {
                System.out.println("- " + ponto.getNome() + " | Preco: " + ponto.getP());
            }
        }
        System.out.println("Melhor ponto para demanda (" + demandaAtual + "):");

        PontoDeColeta melhorPonto = null;
```

```java
        int maiorPreco = -1;

        for (PontoDeColeta ponto : pontos) {
            if ((demandaAtual.equals("VIDRO") && ponto instanceof ColetaVidro) ||
                (demandaAtual.equals("METAL") && ponto instanceof ColetaMetal) ||
                (demandaAtual.equals("PAPELAO") && ponto instanceof ColetaPapelao)) {

                if (ponto.getP() > maiorPreco) {
                    maiorPreco = ponto.getP();
                    melhorPonto = ponto;
                }
            }
        }

        if (melhorPonto != null) {
            System.out.println("- " + melhorPonto.getNome() + " | Preco: " + melhorPonto.getP());
        } else {
            System.out.println("Nenhum ponto disponivel para esta demanda.");
        }

        System.out.println("Pior ponto para demanda (" + demandaAtual + "):");

        PontoDeColeta piorPonto = null;
        int menorPreco = Integer.MAX_VALUE;

        for (PontoDeColeta ponto : pontos) {
            if ((demandaAtual.equals("VIDRO") && ponto instanceof ColetaVidro) ||
                (demandaAtual.equals("METAL") && ponto instanceof ColetaMetal) ||
                (demandaAtual.equals("PAPELAO") && ponto instanceof ColetaPapelao)) {

                if (ponto.getP() < menorPreco) {
                    menorPreco = ponto.getP();
                    piorPonto = ponto;
                }
            }
        }

        if (piorPonto != null) {
            System.out.println("- " + piorPonto.getNome() + " | Preco: " + piorPonto.getP());
        } else {
            System.out.println("Nenhum ponto disponivel para esta demanda.");
        }
    }



    PontoDeColeta buscarEstabelecimento(String nome) {
        for (PontoDeColeta ponto : pontos) {
            if (ponto.getNome().equalsIgnoreCase(nome)) {
                return ponto;
            }
        }
```

```
        System.out.println("Ponto de Coleta nao encontrado.");
        return null;
    }

    private boolean verificarEstabelecimento(PontoDeColeta novoPonto) {
        for (PontoDeColeta ponto : pontos) {
            if (novoPonto.getClass() == ponto.getClass() &&
                novoPonto.getNome().equalsIgnoreCase(ponto.getNome())) {
                return true;
            }
        }
        return false;
    }

}

```

Lista5\Q4\src\Main.java

```java
import java.util.Scanner;

import L5Q4.*;

public class Main {
    public static void main(String[] args) throws Exception {
        System.out.println("Sistema de gerenciamento de coleta de materiais reciclaveis");
        Scanner scanner = new Scanner(System.in);
        System.out.println("Menu de opcoes:");
        System.out.println("1. Cadastrar Estabelecimento");
        System.out.println("2. Cadastrar Material");
        System.out.println("3. Registrar Coleta");
        System.out.println("0. Sair");
        ListaDeEstabelecimentos lista = new ListaDeEstabelecimentos();
        while (true) {
            System.out.println("Escolha uma opcao:");
            int opcao = scanner.nextInt();
            if (opcao == 0) {
                System.out.println("Encerrando o programa...");
                break;
            }
            switch (opcao) {
                case 1:
                    System.out.println("Digite o nome do estabelecimento:");
                    String nomeEstabelecimento = scanner.next();
                    System.out.println("Digite o peso em kilos:");
                    int peso = scanner.nextInt();
                    System.out.println("Digite o tipo de material coletado (VIDRO, PAPELAO, METAL):");
                    String tipoMaterial = scanner.next();
                    PontoDeColeta ponto;
                    if (tipoMaterial.equalsIgnoreCase("VIDRO")) {
                        ponto = new ColetaVidro(nomeEstabelecimento, peso);
                    } else if (tipoMaterial.equalsIgnoreCase("PAPELAO")) {
                        ponto = new ColetaPapelao(nomeEstabelecimento, peso);
                    } else if (tipoMaterial.equalsIgnoreCase("METAL")) {
                        ponto = new ColetaMetal(nomeEstabelecimento, peso);
                    } else {
                        System.out.println("Tipo de material invalido. Tente novamente.");
                        continue;
                    }
                    lista.registrarEstabelecimento(ponto);
                    break;
                case 2:
                    System.out.println("Digite a demanda atual (VIDRO, PAPELAO, METAL):");
                    String demandaAtual = scanner.next();
                    lista.definirDemanda(demandaAtual);
                    break;
                case 3:
                    lista.mostrarLista();
                    break;
                default:
```

```
                    System.out.println("Opcao invalida! Tente novamente.");
            }
        }
        scanner.close();
    }
}

```

Lista5\Q4\src\L5Q4\PontoDeColeta.java

```java
package L5Q4;

public class PontoDeColeta {
    private String nome;
    private int p;


    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public int getP() {
        return p;
    }

    public void setP(int p) {
        this.p = p;
    }


    public PontoDeColeta(String nome, float p) {
        this.nome = nome;
        this.p = (int) p;
    }

    public void definirPreco(int p, String demandaAtual) {
        this.p = (int) (p * 5);
    }
}

```